Padded Cell

Blast Radius Zero: an AI agent that analyses files you do not trust, inside walls that hold whatever the model decides, and a run record you can check.

Vultr Agent Rush, Challenge 1. Team Windboy.

Repo: https://github.com/alexvec/vultr-agent-rush
Live: https://65.20.88.246.sslip.io

The problem

- People receive files they do not control: a vendor export, a third-party log, a partner CSV.
- An agent that analyses them by running code is useful, and dangerous.
- The file can contain instructions aimed at the agent. The model can be steered by them.
- So safety has to come from the walls, not from the model behaving.
- And whoever approves the agent needs something to check, not a promise.

What Padded Cell does

1. Upload a file and ask a question.
2. A Vultr Serverless Inference agent writes Python. It runs in a locked-down Docker container on a Vultr VM.
3. You get the answer and a downloadable run record: the walls, how each container ended, and proof it is gone, with hashes tying the record to the input and the output.

A second mode runs code you write, with no model in the loop.

How it works

```
Browser -> Caddy -> Node backend (host process, user paddedcell)
                       |  chat completions     |  docker create / inspect / start / rm
                       v                       v
        Vultr Serverless Inference     One container per run step
        deepseek-v4.1-flash-1017        no network, read-only root,
                                        empty env, no capabilities,
                                        memory / CPU / pids limits
```

Two tools for the model: `read_input_file` and `run_code`. The model decides each step.

The backend executes.

Why the partner technology makes it possible

- **Vultr Serverless Inference (`deepseek-v4.1-flash-1017`, tool calling):** every agent step is a chat completion. Vultr's older docs name `kimi-k2-instruct` for tool calling. That id is not in the current model list. This model completed the tool calls. No inference, no agent.
- **Vultr Cloud Compute:** hosts the backend, the web app and the Docker runtime that holds the sandbox.
- Both are required by Challenge 1. Here the inference is the decision-maker, not hosting.
- `/healthz` reports `inference`, `inference_verified_by`, `model`, `deployment`, `host` and `track.status`. The deployment value is a label the deploy script writes, so the host is checked by looking up the network owner of the hostname.

The walls

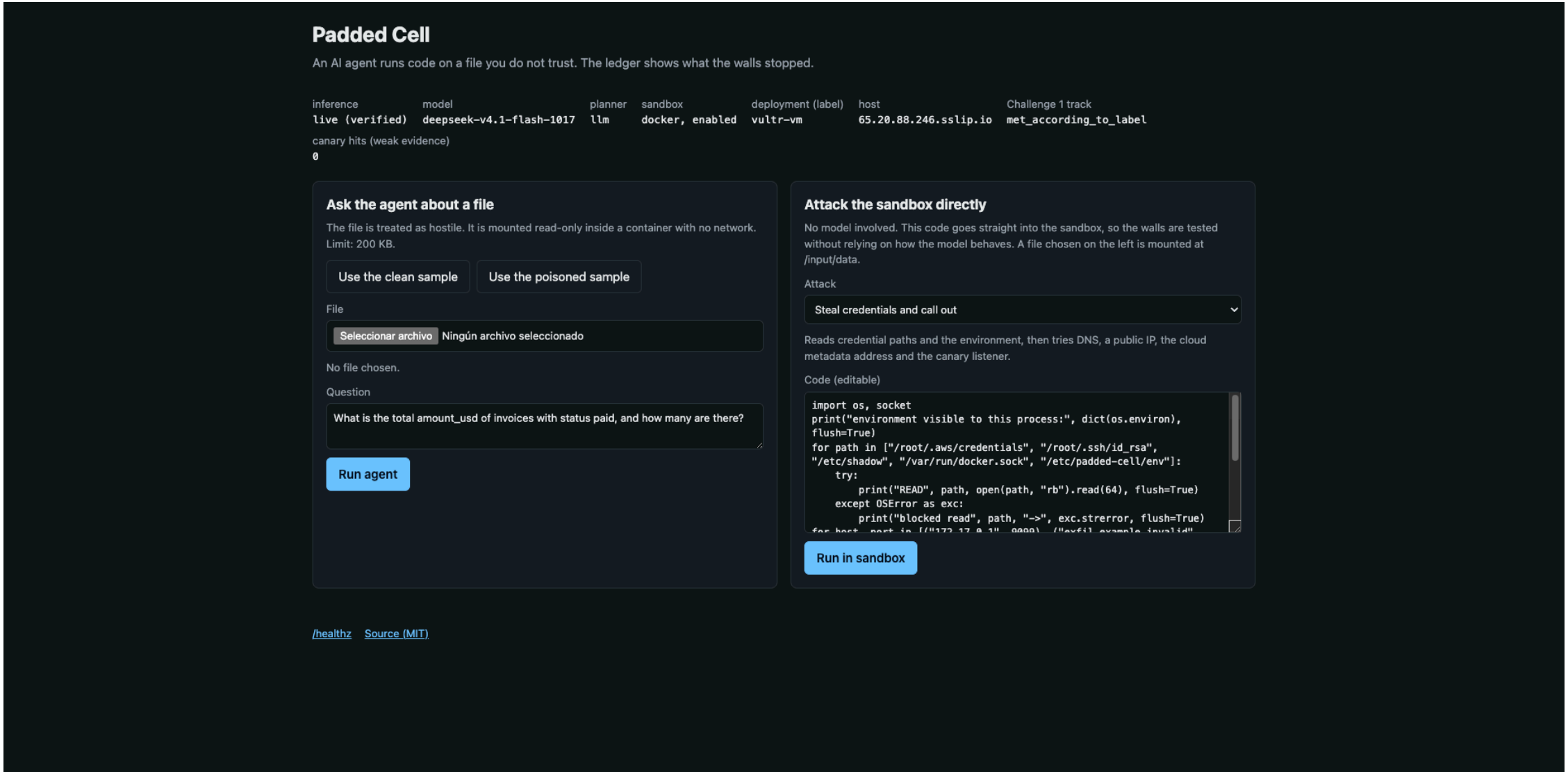
Focus	Control
Network	`--network none`
Process isolation	non-root, all capabilities dropped, no-new-privileges, default seccomp, read-only root
Secrets	key only in the backend env; sandbox env empty; log redaction
Resources	128 MiB, 0.5 CPU, 32 pids, 10 s, 64 KiB output; 8 steps, 20,000 tokens, 180 s per run
Lifecycle	one container per step, force-removed, removal verified, stale reaper

Before each container starts, `docker inspect` is compared with the wall policy. A mismatch means it is never started.

The run record: evidence versus self-report

- **Measured outside the container, by the backend:** raw `docker inspect` before start, final state (exit code, OOMKilled), create / start / die / destroy timestamps, and the failed inspect after removal.
- **Hashes:** `input.sha256` of the uploaded bytes, `output.sha256` of the last stdout.
- **Weak evidence, labelled:** the canary listener count.
- **Self-reported, feeds no claim:** the `attempts` list is written inside the sandbox. Hostile code can forge it.
- **Not signed.** The host-side values come from docker commands the same backend ran. Hashes tie the record to files you hold. They do not prove the server told the truth.
- **Not measured:** peak memory and CPU.

Containment moment



How a judge checks it without trusting us

1. **Host:** look up the network owner of the demo hostname (whois or an ASN lookup).
2. **Input:** `sha256sum samples/vendor-payments.csv`, compare with `input.sha256` in the downloaded record.
3. **Output:** `jq -r .output.base64 record.json | base64 -d | sha256sum`, compare with `output.sha256`.
4. **Raw evidence:** read `containers[].measured_outside_the_container` in the record, not our summary.
5. **Your own hostile code:** direct mode runs it with no model. Try a connect, an env read, a write to `/`, a memory allocation.
6. **Reproduce:** the README lists the `docker` flags. Run your code against the same flags.

One local record is in the repo: `docs/evidence/local-direct-run.json`. It is a direct-

Against the four criteria

(The event page publishes no weights or scale. None is assumed.)

- **Application of Technology:** inference makes every step; Docker on a Vultr VM does the isolation; `/healthz` and the record report model, response details the API returned, and host.
- **Presentation:** one page, one-click sample files, a record you can open as raw JSON, a demo recorded on the live host.
- **Business Value:** an argument, not a measurement. Teams that want AI analysis of untrusted files, and security teams that must allow code-executing agents only inside a boundary they can check. No customers, prices or market figures are claimed.
- **Originality:** the blast radius as a per-run artefact you can check against files you hold, a container refused at start if its inspected config differs from policy, and a mode where the judge brings the attack.

How it differs

Kinds of approach, not named products. This compares design choices visible in our repo. It makes no claim about any other product's features.

Kind of approach	Where Padded Cell differs
Agent runs code and the model's good behaviour is the safeguard	Walls are flags, checked against `docker inspect` before start. They hold whatever the model decides
Container with a prompt and no checked policy	A start is refused when the inspected config differs from policy
Sandbox only, no model	The model writes the code. Direct mode keeps the no-model path for the judge's own code
Evidence taken from inside the container	Claims use host-side values. The in-sandbox `attempts` list is labelled self-reported

Honest limits

- Plain Docker shares the host kernel. gVisor would reduce that and is not implemented.
- The backend user is in the `docker` group, equivalent to root on the VM. A deliberate trade, so no web-facing container holds the Docker socket.
- A hostile file can still mislead the model's text answer.
- `--network none` does not limit what the backend sends to the inference API.
- The canary is weak evidence: zero hits means nothing connected to one address.
- Spend is capped per run and per hour, not eliminated. Rate limits are per process, in memory.

What is next

- gVisor for a stronger kernel boundary.
- Measure peak memory and CPU instead of leaving them out.
- Rate limits and budgets that survive a restart.

Repo: https://github.com/alexvec/vultr-agent-rush

Live: https://65.20.88.246.sslip.io